OS ORANGE LEVEL : UNIT 4

Name: Muhammad Affan Dumba
SRN: PES1UG24CS280
Sec: E

SCREENSHOT 1A:

```
 127 affan@affan-VirtualBox:~/cs280-pes-vcs$ nano object.c
affan@affan-VirtualBox:~/cs280-pes-vcs$ make clean
make test_objects
./test_objects
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘compute_hash’:
object.c:41:5: warning: ‘SHA256_Init’ is deprecated: Since OpenSSL 3.0 [-Wdeprecated-declarations]
   41 |     SHA256_Init(&ctx);
      |     ^~~~~~~~~~~
In file included from object.c:18:
/usr/include/openssl/sha.h:73:27: note: declared here
   73 | OSSL_DEPRECATEDIN_3_0 int SHA256_Init(SHA256_CTX *c);
      |                           ^~~~~~~~~~~
object.c:42:5: warning: ‘SHA256_Update’ is deprecated: Since OpenSSL 3.0 [-Wdeprecated-declarations]
   42 |     SHA256_Update(&ctx, data, len);
      |     ^~~~~~~~~~~~~
/usr/include/openssl/sha.h:74:27: note: declared here
   74 | OSSL_DEPRECATEDIN_3_0 int SHA256_Update(SHA256_CTX *c,
      |                           ^~~~~~~~~~~~~
object.c:43:5: warning: ‘SHA256_Final’ is deprecated: Since OpenSSL 3.0 [-Wdeprecated-declarations]
   43 |     SHA256_Final(id_out->hash, &ctx);
      |     ^~~~~~~~~~~~
/usr/include/openssl/sha.h:76:27: note: declared here
   76 | OSSL_DEPRECATEDIN_3_0 int SHA256_Final(unsigned char *md, SHA256_CTX *c);
      |                           ^~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

SCREENSHOT 1B:

```
affan@affan-VirtualBox:~/cs280-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

SCREENSHOT 2A:

```
Apr 21 12:33
Terminal
        @echo "=== Running integration tests ==="
        bash test_sequence.sh
affan@affan-VirtualBox:~/cs280-pes-vcs$ nano Makefile
affan@affan-VirtualBox:~/cs280-pes-vcs$ make clean
make test_tree
./test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘compute_hash’:
object.c:41:5: warning: ‘SHA256_Init’ is deprecated: Since OpenSSL 3.0 [-Wdeprecated-declarations]
   41 |     SHA256_Init(&ctx);
      |     ^~~~~~~~~~~
In file included from object.c:18:
/usr/include/openssl/sha.h:73:27: note: declared here
   73 | OSSL_DEPRECATEDIN_3_0 int SHA256_Init(SHA256_CTX *c);
      |                           ^~~~~~~~~~~
object.c:42:5: warning: ‘SHA256_Update’ is deprecated: Since OpenSSL 3.0 [-Wdeprecated-declarations]
   42 |     SHA256_Update(&ctx, data, len);
      |     ^~~~~~~~~~~~~
/usr/include/openssl/sha.h:74:27: note: declared here
   74 | OSSL_DEPRECATEDIN_3_0 int SHA256_Update(SHA256_CTX *c,
      |                           ^~~~~~~~~~~~~
object.c:43:5: warning: ‘SHA256_Final’ is deprecated: Since OpenSSL 3.0 [-Wdeprecated-declarations]
   43 |     SHA256_Final(id_out->hash, &ctx);
      |     ^~~~~~~~~~~~
/usr/include/openssl/sha.h:76:27: note: declared here
   76 | OSSL_DEPRECATEDIN_3_0 int SHA256_Final(unsigned char *md, SHA256_CTX *c);
      |                           ^~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function ‘write_tree_recursive’:
tree.c:147:13: warning: ‘__builtin___strncpy_chk’ output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-trun
cation]
  147 |             strncpy(te->name, dir_name, sizeof(te->name) - 1);
      |             ^
gcc -Wall -Wextra -O2 -c index.c -o index.o
```

```
Apr 21 12:34
Terminal
gcc -Wall -Wextra -O2 -c index.c -o index.o
index.c: In function ‘index_load’:
index.c:130:47: warning: format ‘%zu’ expects argument of type ‘size_t *’, but argument 6 has type ‘uint32_t *’ {aka ‘unsigned int *’} [
-Wformat=]
  130 |         int result = fscanf(f, "%o %64s %ld %zu %255s",
      |                                               ~~^
      |                                                 |
      |                                                 long unsigned int *
      |                                               %u
  131 |                             &e->mode, hex,
  132 |                             &e->mtime_sec, &e->size,
      |                                            ~~~~~~~~
      |                                            |
      |                                            uint32_t * {aka unsigned int *}
index.c: In function ‘index_save’:
index.c:163:33: warning: format ‘%zu’ expects argument of type ‘size_t’, but argument 6 has type ‘uint32_t’ {aka ‘unsigned int’} [-Wform
at=]
  163 |         fprintf(f, "%o %s %ld %zu %s\n",
      |                                ~~^
      |                                  |
      |                                  long unsigned int
      |                                %u
  164 |                 e->mode, hex, e->mtime_sec, e->size, e->path);
      |                                             ~~~~~~~
      |                                             |
      |                                             uint32_t {aka unsigned int}
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
affan@affan-VirtualBox:~/cs280-pes-vcs$
```

SCREENSHOT 2B:

```
affan@affan-VirtualBox:~/cs280-pes-vcs$ find .pes/objects -type f
affan@affan-VirtualBox:~/cs280-pes-vcs$ make pes
./pes init
echo "hello world" > file1.txt
./pes add file1.txt
find .pes/objects -type f
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Segmentation fault (core dumped)
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
affan@affan-VirtualBox:~/cs280-pes-vcs$ xxd .pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
```

SCREENSHOT 3A:

```
      |                                   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                   __va_arg_pack ());
      |                                   ~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
affan@affan-VirtualBox:~/cs280-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
affan@affan-VirtualBox:~/cs280-pes-vcs$ echo "Hello" > hello.txt
affan@affan-VirtualBox:~/cs280-pes-vcs$ echo "World" > file2.txt
affan@affan-VirtualBox:~/cs280-pes-vcs$ ./pes add hello.txt file2.txt
affan@affan-VirtualBox:~/cs280-pes-vcs$ ./pes status
Staged changes:
    staged: file2.txt
    staged: hello.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    untracked: index.c
    untracked: commit.h
    untracked: file1.txt
    untracked: README.md
    untracked: index.h
    untracked: Makefile
    untracked: pes.h
    untracked: test_tree.c
    untracked: tree.c
    untracked: commit.c
    untracked: tree.h
    untracked: pes.c
    untracked: test_sequence.sh
    untracked: test_objects.c
    untracked: object.c
    untracked: .gitignore
```

SCREENSHOT 3B:

```
affan@affan-VirtualBox:~/cs280-pes-vcs$ cat .pes/index
100644 b3d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12 1776780393 6 file2.txt
100644 66224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 1776780384 6 hello.txt
```

SCREENSHOT 4A:

```
affan@affan-VirtualBox:~/cs280-pes-vcs$ cat .pes/index
100644 b3d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12 1776780393 6 file2.txt
100644 66224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 1776780384 6 hello.txt
affan@affan-VirtualBox:~/cs280-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 86afd56f9352... Initial commit
affan@affan-VirtualBox:~/cs280-pes-vcs$ echo "World 2" >> hello.txt
./pes add hello.txt
./pes commit -m "Update hello"
Committed: c86a009a3b60... Update hello
affan@affan-VirtualBox:~/cs280-pes-vcs$ echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: fd7717d81963... Add farewell
affan@affan-VirtualBox:~/cs280-pes-vcs$ ./pes log
commit fd7717d819636c48b3934872b657128dbe66dfb7a9386c3239aef68884c92ee2
Author: PES User <pes@localhost>
Date:   1776780821

    Add farewell

commit c86a009a3b6023dd2ed18254c0b7c3fab499fee06f4c8b6639f6122a03539663
Author: PES User <pes@localhost>
Date:   1776780809

    Update hello

commit 86afd56f9352d48c73c9e1abd6f1c975c8d5c6891094202c89df3f6c2fc6a7c5
Author: PES User <pes@localhost>
Date:   1776780800

    Initial commit
```

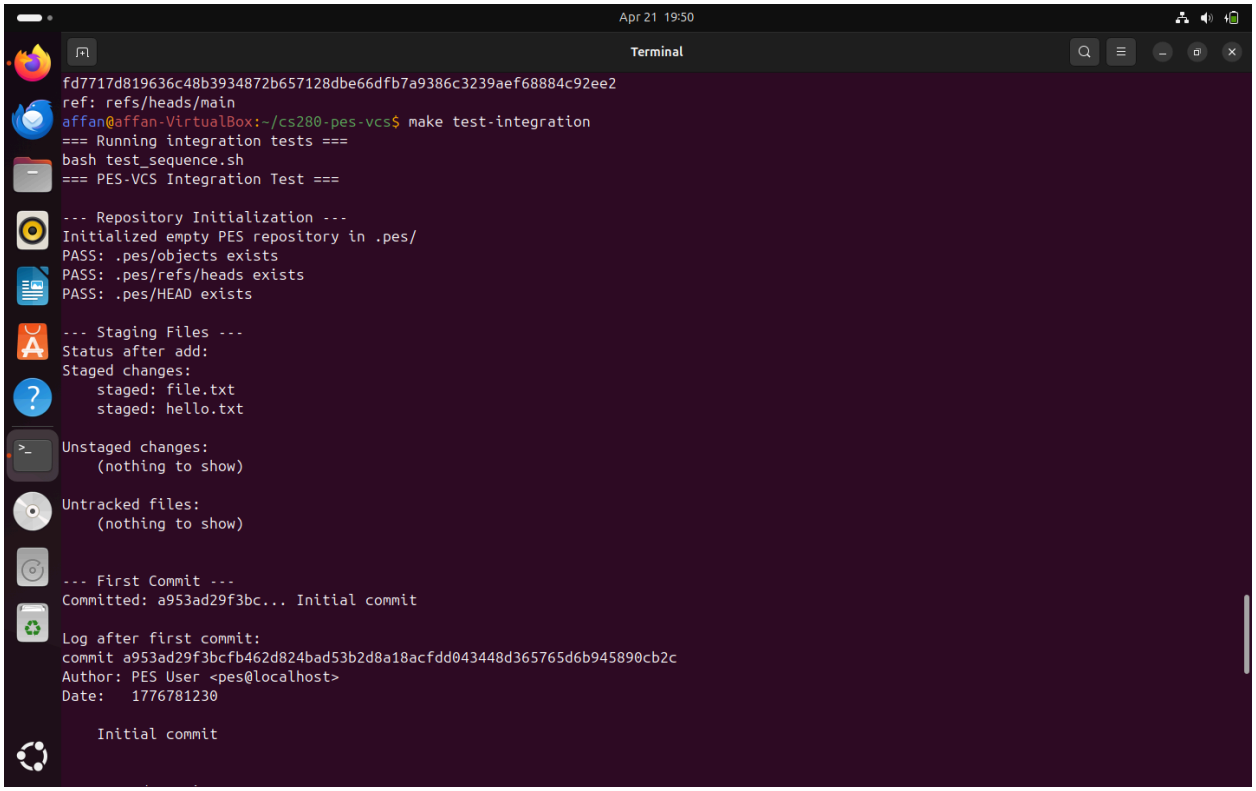
SCREENSHOT 4B:

```
affan@affan-VirtualBox:~/cs280-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2b/46ba37ae5408380bd2b2bab44de4c84c4a0c2cec280911abccc419344e0322
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7a/e8688a81dfae3e31ee6e01948e8ce047b7a364ea413cf5b1dba7925c8ed841
.pes/objects/80/409e907d532a7b0a7353195d616fb3991bb52d25c1497ed9284249cc967447
.pes/objects/86/afd56f9352d48c73c9e1abd6f1c975c8d5c6891094202c89df3f6c2fc6a7c5
.pes/objects/94/ab0bc1df90f7b7acc8c6efbbddad7f7a7abc78b5f170b344169c3267eee948
.pes/objects/b3/d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12
.pes/objects/c8/6a009a3b6023dd2ed18254c0b7c3fab499fee06f4c8b6639f6122a03539663
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/fd/7717d819636c48b3934872b657128dbe66dfb7a9386c3239aef68884c92ee2
.pes/refs/heads/main
```

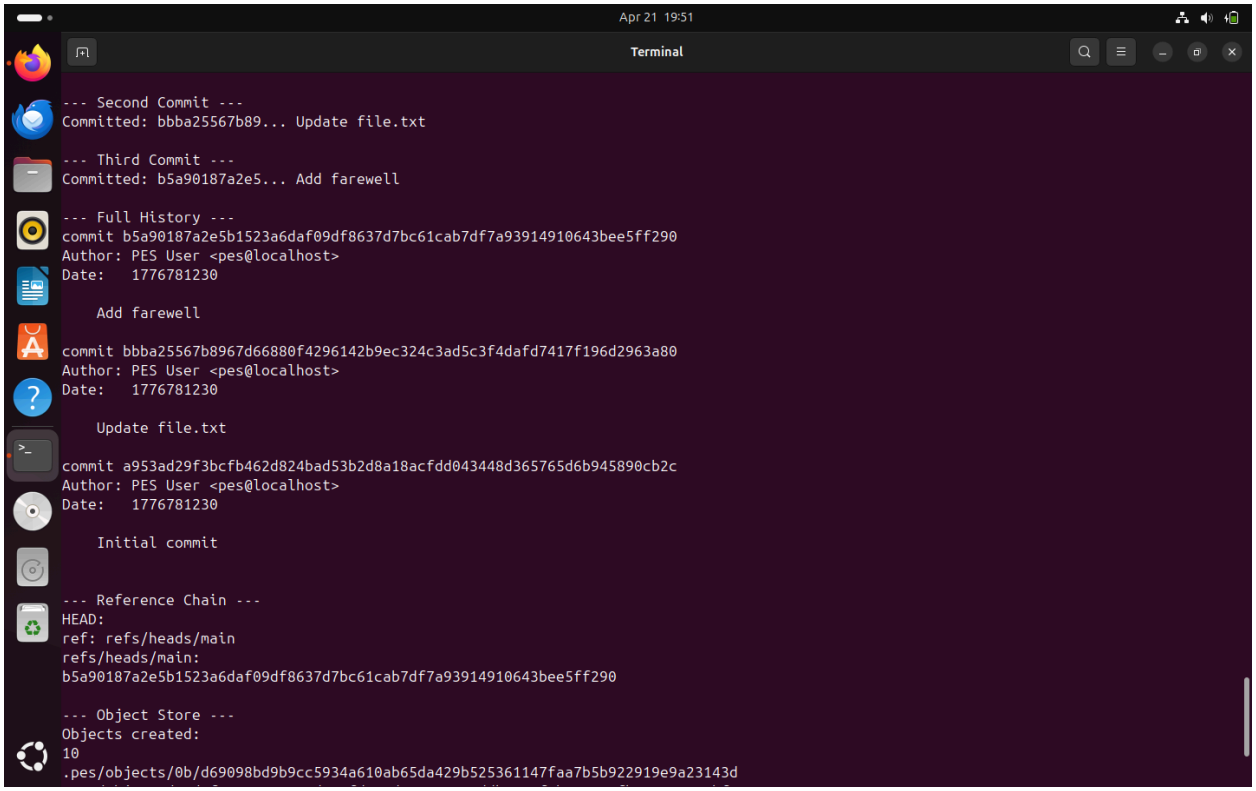
SCREENSHOT 4C:

```
affan@affan-VirtualBox:~/cs280-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
fd7717d819636c48b3934872b657128dbe66dfb7a9386c3239aef68884c92ee2
ref: refs/heads/main
```

INTEGRATION TEST:



```
fd7717d819636c48b3934872b657128dbe66dfb7a9386c3239aef68884c92ee2
ref: refs/heads/main
affan@affan-VirtualBox:~/cs280-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
    staged: file.txt
    staged: hello.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    (nothing to show)


--- First Commit ---
Committed: a953ad29f3bc... Initial commit

Log after first commit:
commit a953ad29f3bcfb462d824bad53b2d8a18acfdd043448d365765d6b945890cb2c
Author: PES User <pes@localhost>
Date:   1776781230

    Initial commit
```



```
--- Second Commit ---
Committed: bbba25567b89... Update file.txt

--- Third Commit ---
Committed: b5a90187a2e5... Add farewell

--- Full History ---
commit b5a90187a2e5b1523a6daf09df8637d7bc61cab7df7a93914910643bee5ff290
Author: PES User <pes@localhost>
Date:   1776781230

    Add farewell

commit bbba25567b8967d66880f4296142b9ec324c3ad5c3f4dafd7417f196d2963a80
Author: PES User <pes@localhost>
Date:   1776781230

    Update file.txt

commit a953ad29f3bcfb462d824bad53b2d8a18acfdd043448d365765d6b945890cb2c
Author: PES User <pes@localhost>
Date:   1776781230

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
b5a90187a2e5b1523a6daf09df8637d7bc61cab7df7a93914910643bee5ff290

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/a9/53ad29f3bcfb462d824bad53b2d8a18acfdd043448d365765d6b945890cb2c
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b5/a90187a2e5b1523a6daf09df8637d7bc61cab7df7a93914910643bee5ff290
.pes/objects/bb/ba25567b8967d66880f4296142b9ec324c3ad5c3f4dafd7417f196d2963a80
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

To switch branches, the first thing we need to do is update the `.pes/HEAD` file so it points to the new branch. For example if we run `pes checkout feature`, we update HEAD to say `ref: refs/heads/feature`. This part is simple since it is just writing to a file.

The more involved part is syncing the working directory. We have to read the target branch's latest commit, find the tree it points to, walk through all the files in that tree, and write each file's contents to disk. Any file that exists in the current branch but not in the target branch needs to be deleted.

What makes this tricky in practice is dealing with user changes. If the user has edited a file that is different between the two branches, we cannot just overwrite it since that would destroy their work. So before making any changes, we need to go through every file that would be affected and

check whether it has been modified. If it has, we stop and show an error. Another edge case is untracked files. If the user has a new file sitting in the working directory with the same name as a file in the target branch, we also need to stop rather than silently overwrite it. On top of all this, the operation is not atomic at the filesystem level, so a crash halfway through leaves the repo in a broken state which is hard to recover from.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

The index already stores everything we need to do this check. For each staged file it keeps the blob hash, the file size, and the last modification time recorded at the time of staging. To check if a file is dirty, we look at its current size and modification time on disk using stat and compare them against what the index has stored. If they are the same, the file has not been touched since it was last staged and it is safe to overwrite. If they differ, the file has been modified since staging which means it has unsaved changes. We also need to check whether the file actually changes between branches. We do this by comparing the blob hash in the current index entry against the blob hash in the target branch's tree for the same path. If the hashes are different, the file would be changed by the checkout. If both conditions are true at the same time, meaning the file is modified in the working directory and also differs between the two branches, then we know a checkout would destroy unsaved work. At that point we refuse and print an error telling the user to commit or stash their changes first. The whole thing only needs the index and a couple of hash lookups, no actual diffing of file contents required.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Normally HEAD contains something like `ref: refs/heads/main` which means it points to a branch, and that branch pointer moves forward every time you commit. In detached HEAD state, HEAD contains a raw commit hash directly. There is no branch involved.

If you make commits while detached, they get written to the object store correctly and HEAD keeps getting updated to the new commit hash. So the commits do exist and are valid objects on disk. The problem is that once you switch to a different branch, HEAD gets updated to point to that branch instead. Your detached commits are now orphaned because nothing in the refs directory points to them anymore. They are still physically on disk but nothing references them, so a garbage collection run would eventually delete them.

To get them back, you need the hash of the last commit you made while detached. If you ran `pes log` before switching away you can find it there, or you might have it in your terminal history. Once you have the hash you can create a new branch by writing it into `.pes/refs/heads/recovered` manually. After that the commits are reachable through the new branch and are safe from garbage collection.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to. Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

The algorithm follows a mark and sweep approach similar to how garbage collectors work in programming languages.

The mark phase starts by reading every file in `.pes/refs/heads/` to get the tip commit of each branch. For each commit hash found, it gets added to a reachable set. We then follow the parent pointer to the previous commit and keep walking back until we reach the first commit with no parent. For every commit visited we also read its tree, walk the tree recursively, and add every tree hash and blob hash to the reachable set. This gets repeated for all 50 branches.

The sweep phase then runs `find .pes/objects -type f` to list every object stored on disk. For each one we reconstruct its hash from the path and check if it is in the reachable set. Anything not in the set is unreachable and gets deleted.

For the data structure a hash set is the best choice because it gives constant time lookups during the sweep. A sorted array with binary search would also work.

For the estimate, assuming around 10 files per commit on average, each commit generates roughly 10 blobs, 3 trees and 1 commit object which is about 14 objects. With 100,000 commits that comes to around 1.4 million objects to visit during the mark phase. This would run in a few seconds on modern hardware with a good hash set implementation.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running garbage collection at the same time as a commit is dangerous because there is a window of time where a new object exists on disk but is not yet referenced by anything. Here is exactly how the race plays out.

The commit operation starts and writes a new blob to `.pes/objects/`. At this moment the blob is on disk but no tree or commit points to it yet. The GC then starts its mark phase and walks all reachable objects from the branch heads. Since nothing references the new blob yet it does not get marked as reachable. The GC moves to the sweep phase and deletes the blob because it looks like an orphan. The commit operation then continues and tries to build a tree that includes the blob it wrote. It creates the tree and commit objects and updates HEAD. But the blob has already been deleted. The repository is now broken because the latest commit points to a tree that references a blob that no longer exists on disk.

Real Git avoids this with a simple but effective strategy. It applies a grace period of two weeks to all loose objects. Any object created within the last two weeks is never deleted during GC regardless of whether it appears reachable or not. This means a freshly written blob from a concurrent commit is always protected because it will be newer than the grace period threshold. Git also writes a lock file called `gc.pid` when GC starts so that only one GC process can run at a time preventing two garbage collection runs from interfering with each other.